A ZERO-TO-MASTER LEARNING BOOK

React & React Native

From “What is a component?” to shipping apps on the Play Store and App Store

Explained like you are 5 • Built up to professional depth

React 19 • Expo Router • New Architecture • TypeScript

Edition 1 — released one Part at a time

Table of Contents

Front matter

- How to Use This Book

Web: React

- **Part 0** – JavaScript Needed for React *(included in this edition)*
- **Part 1** – React Basics
- **Part 2** – State and Events
- **Part 3** – Hooks Deep Dive
- **Part 4** – How React Works Internally
- **Part 5** – Routing
- **Part 6** – Data Fetching and APIs
- **Part 7** – State Management
- **Part 8** – Forms and Validation
- **Part 9** – Performance
- **Part 10** – Advanced Patterns
- **Part 11** – TypeScript with React
- **Part 12** – Testing
- **Part 13** – Next.js Essentials

Mobile: React Native

- **Part 14** – React Native Basics
- **Part 15** – React Native Navigation
- **Part 16** – React Native Data and Storage
- **Part 17** – Device Features
- **Part 18** – UI and Animations
- **Part 19** – RN Performance and Native Code
- **Part 20** – Build and Publish

Capstone

- **Part 21** – Master Projects (Task Manager, Expense Tracker, Monorepo)

Appendix

- 50 Interview Questions, Hooks Cheat Sheet, Recommended Folder Structure, Glossary

Part 0 in detail

#	Topic
0.1	`let` and `const`
0.2	Arrow functions
0.3	Template literals
0.4	Destructuring
0.5	Spread and rest
0.6	Array method: `map`
0.7	Array method: `filter`
0.8	Array method: `reduce`
0.9	Array method: `find` (and friends)
0.10	Modules: `import` and `export`
0.11	Promises
0.12	`async` / `await`
0.13	Optional chaining `?.`
0.14	Nullish coalescing `??`
	Quiz and Mini Project

How to Use This Book

Who this book is for

You know a little JavaScript. You want to build real apps with React (web) and React Native (mobile). This book treats **every topic as new**, so nothing is skipped.

How every topic is built

Each topic follows the same 8 steps. Once you see the pattern, you can read fast.

Step	Name	What it gives you
1	**Title**	A number, like `3.2 useEffect`
2	**ELI5**	A real-life picture (kitchen, school, shop, toys)
3	**Real explanation**	What it is, why it exists, how it works inside
4	**Example code**	A complete, commented example you can run
5	**Output**	What you see when you run it
6	**Common mistakes**	2–3 traps with wrong vs right code
7	**Mini practice**	One small exercise to try yourself
8	**Key takeaway**	One line to remember

At the end of every Part you get a **5-question quiz** (with answers) and a **mini project** that mixes the whole Part.

The best way to learn

1. **Read** the ELI5 first. If it clicks, the rest is easy.
2. **Type** the example yourself. Do not copy-paste. Typing builds memory.
3. **Break** it on purpose. Change a line and predict what happens.
4. **Do** the mini practice task before you move on.
5. **Take** the quiz without looking at the answers.
6. **Build** the mini project. This is where learning sticks.

A good pace is **one Part per week**, about 1 hour a day.

What you need installed

Tool	Why	Check with
Node.js 20+	Runs JavaScript outside the browser	`node --version`
A code editor	VS Code is a good choice	–
A browser	Chrome or Firefox, with DevTools	–
Git	Saves your work	`git --version`

Later Parts add more tools (Vite, Expo, Android Studio, Xcode). Each Part tells you exactly what to install.

Code conventions

- Modern JavaScript only. React 19, function components, hooks. No class components (one short “legacy” note later).
- React Native examples work with **Expo**.
- `// ❌` marks wrong code. `// ✅` marks right code.
- Part 0 examples run with Node. Save as `example.js` and run `node example.js`.
- ASCII diagrams show flows. Tables compare ideas.

PART 0 – JavaScript Needed for React

React is “just JavaScript”. If these 14 ideas feel easy, React will feel easy too. You will see them in **every** React file you ever read.

> **Run the examples:** save each `js` example as `example.js` and run `node example.js`.

0.1 `let` and `const`

ELI5

Imagine a **lunch box with a name sticker**. With `const`, the sticker is glued on forever: this box is always “Asha’s box”. You can still change the food *inside*. With `let`, the sticker is a whiteboard label: you can erase it and write a new name.

REAL EXPLANATION

- `const` creates a name that **cannot point to a new value**. It does *not* freeze objects or arrays. You can still change what is inside them.
- `let` creates a name you can **reassign**.
- Both are **block scoped**: they live only inside the `{ }` where you made them.
- `var` is the old way. It ignores blocks and causes bugs. **Do not use `var`.**
- Rule of thumb in React: use `const` by default. Use `let` only when you must reassign.

Keyword	Reassign?	Block scoped?	Use today?
`const`	No	Yes	Yes (default)
`let`	Yes	Yes	Yes (when needed)
`var`	Yes	No (function)	No

EXAMPLE CODE

```
// const: the name cannot be reassigned
const name = "Asha";
// name = "Ravi";   // would throw: TypeError (assignment to constant)

// const with an array: we can change the INSIDE, not the label
const cart = ["apple"];
cart.push("banana");
console.log(cart);

// let: can be reassigned
let score = 0;
score = score + 10;
console.log(score);

// block scope: "inside" only exists in these braces
if (true) {
  let inside = "I live in the block";
  console.log(inside);
}
console.log(typeof inside); // "undefined" because inside is gone here
```

OUTPUT / WHAT HAPPENS

```
[ 'apple', 'banana' ]
10
I live in the block
undefined
```

The array changed even though it is `const`. The variable `inside` is not visible outside the `if` block.

COMMON MISTAKES

- **Reassigning a `const`.**

```
// ❌ Wrong
const count = 0;
count = count + 1;   // TypeError

// ✅ Right
let count2 = 0;
count2 = count2 + 1;
```

- **Thinking `const` makes objects frozen.**

```
// ❌ Wrong belief: "user can't change"
const user = { name: "Asha" };
user.name = "Ravi";  // this works!

// ✅ Right: if you need it frozen
const frozen = Object.freeze({ name: "Asha" });
```

- **Using `var` in loops.** `var` leaks out of the loop. Use `let`.

MINI PRACTICE TASK

Create `const PI = 3.14`. Create `let radius = 2`. Print the area (`PI * radius * radius`). Change `radius` to 5 and print again.

KEY TAKEAWAY

> Use `const` by default, `let` when you must reassign, and never `var`.

0.2 Arrow functions

ELI5

A normal function is a **recipe card with a title**. An arrow function is a **sticky note** with the same recipe, written short. Same cooking, less writing.

REAL EXPLANATION

An arrow function is a shorter way to write a function. It uses `=>`.

- One expression and no `{ }`: the result is **returned automatically** (implicit return).
- With `{ }`: you must write `return` yourself.
- One parameter: the `( )` are optional. Zero or many parameters: `( )` are needed.
- Returning an **object** directly? Wrap it in `( )`.
- Arrow functions do not have their own `this`. This is why React uses them for event handlers and callbacks.

Every React component and every callback you write will be an arrow function or a normal function.

EXAMPLE CODE

```
// 1. Normal function
function addOld(a, b) {
  return a + b;
}

// 2. Arrow function with a body
const add = (a, b) => {
  return a + b;
};

// 3. Arrow function with implicit return (no braces, no return)
const double = (n) => n * 2;

// 4. Returning an object: wrap it in parentheses
const makeUser = (name) => ({ name, active: true });

// 5. Used as a callback
const numbers = [1, 2, 3];
const doubled = numbers.map((n) => n * 2);

console.log(addOld(2, 3));
console.log(add(2, 3));
console.log(double(4));
console.log(makeUser("Asha"));
console.log(doubled);
```

OUTPUT / WHAT HAPPENS

```
5
5
8
{ name: 'Asha', active: true }
[ 2, 4, 6 ]
```

COMMON MISTAKES

- **Forgetting `return` when you use braces.**

```
// ❌ Wrong: returns undefined
const triple = (n) => { n * 3 };

// ✅ Right
const triple2 = (n) => { return n * 3; };
const triple3 = (n) => n * 3;
```

- **Returning an object without parentheses.** JavaScript thinks `{ }` is a function body.

```
// ❌ Wrong: returns undefined
const makeBad = (name) => { name };

// ✅ Right
const makeGood = (name) => ({ name });
```

MINI PRACTICE TASK

Write an arrow function `isEven(n)` that returns `true` when `n` is even. Test it with 4 and 7.

KEY TAKEAWAY

> `(args) => result` is a short function. Braces need `return`; parentheses around `{}` return an object.

0.3 Template literals

ELI5

Think of a **fill-in-the-blank card**: “Hello, ____! You have ____ new toys.” You write the card once and drop the words in the blanks. Backticks let you do that in code.

REAL EXPLANATION

A template literal is a string written with **backticks** `` ` ``. Inside it you can:

- put any JavaScript expression with `${ ... }`
- write text over **many lines** without `\n`

Old way: `"Hello, " + name + "!"`. It gets messy fast. In React you use `${ }` for class names, URLs and messages all the time.

EXAMPLE CODE

```
const name = "Asha";
const items = 3;
const price = 49.5;

// Insert variables and expressions
const message = `Hello, ${name}! You have ${items} items.`;
const total = `Total: ${(items * price).toFixed(2)} rupees`;

// Multi-line text
const card = `Dear ${name},
Thank you for shopping.
Bye!`;

// A common React use: building a URL
const userId = 42;
const url = `https://api.example.com/users/${userId}`;

console.log(message);
console.log(total);
console.log(card);
console.log(url);
```

OUTPUT / WHAT HAPPENS

```
Hello, Asha! You have 3 items.
Total: 148.50 rupees
Dear Asha,
Thank you for shopping.
Bye!
https://api.example.com/users/42
```

COMMON MISTAKES

- **Using quotes instead of backticks.**

```
// ❌ Wrong: prints the text ${name} literally
const bad = "Hello ${name}";

// ✅ Right
const good = `Hello ${name}`;
```

- **Putting a statement inside `${ }`.** Only expressions work (things that produce a value).

```
// ❌ Wrong
const x1 = `Result: ${ if (true) { "yes" } }`;

// ✅ Right: use a ternary
const x2 = `Result: ${true ? "yes" : "no"}`;
```

MINI PRACTICE TASK

Make variables `product` and `qty`. Print: `You bought 2 pens.` using a template literal.

KEY TAKEAWAY

> Use backticks and `${ }` to put values inside text.

0.4 Destructuring

ELI5

You get a **toy box** with a car, a ball and a doll inside. Instead of saying “take the car from the box, take the ball from the box…”, you say **“unpack the box: give me car and ball”**. Destructuring unpacks.

REAL EXPLANATION

Destructuring pulls values out of **objects** or **arrays** into separate variables, in one line.

- **Object destructuring** uses `{ }` and matches by **name**.
- **Array destructuring** uses `[ ]` and matches by **position**.

- You can set **default values** and **rename** variables.
- You can destructure **function parameters**. React does this for props: `function Card({ title, price })`.
- `useState` returns an array, so you will write `const [count, setCount] = useState(0)` (array destructuring).

EXAMPLE CODE

```
const user = { name: "Asha", age: 25, city: "Chennai" };

// Object destructuring (by name)
const { name, age } = user;

// Rename and default value
const { city: town, country = "India" } = user;

// Array destructuring (by position)
const colors = ["red", "green", "blue"];
const [first, second] = colors;

// Skip an item with an empty slot
const [, , third] = colors;

// Destructure function parameters (just like React props)
const greet = ({ name, age }) => `${name} is ${age}`;

console.log(name, age);
console.log(town, country);
console.log(first, second, third);
console.log(greet(user));
```

OUTPUT / WHAT HAPPENS

```
Asha 25
Chennai India
red green blue
Asha is 25
```

COMMON MISTAKES

- **Wrong brackets.** Objects use `{ }`, arrays use `[ ]`.

```
// ❌ Wrong: array brackets on an object
const [name] = { name: "Asha" };   // TypeError

// ✅ Right
const { name: n } = { name: "Asha" };
```

- **Wrong name.** Object keys must match exactly.

```
// ❌ Wrong: key is "name", not "username"
const { username } = { name: "Asha" };   // username is undefined

// ✅ Right
const { name: username2 } = { name: "Asha" };
```

MINI PRACTICE TASK

Given `const book = { title: "Atomic Habits", pages: 320 }`, destructure `title` and `pages` and print them.

KEY TAKEAWAY

> `{ }` unpacks objects by name, `[ ]` unpacks arrays by position.

0.5 Spread and rest (. . .)

ELI5

Spread is like **pouring all the sweets from one jar into a new jar**, and maybe adding a few more. **Rest** is the opposite: **“put all the leftover sweets into one bag”**. Same three dots, two jobs.

REAL EXPLANATION

The three dots `...` do two jobs, depending on where you put them:

Where	Name	What it does
In an array/object **literal** or function **call**	Spread	Expands items out
In a function **parameter** or **destructuring**	Rest	Collects leftovers into one

Spread **copies** (a shallow copy). It is the main way to update state in React **without changing the original**: `setUser({ ...user, age: 26 })`. You will use this daily.

EXAMPLE CODE

```
// SPREAD with arrays
const a = [1, 2];
const b = [...a, 3, 4];          // new array: [1,2,3,4]

// SPREAD with objects (later keys win)
const user = { name: "Asha", age: 25 };
const older = { ...user, age: 26 };   // copy, then override age

// SPREAD in a function call
const nums = [5, 1, 9];
console.log(Math.max(...nums));

// REST in parameters: collects all arguments
const sum = (...values) => values.reduce((t, v) => t + v, 0);

// REST in destructuring: "everything else"
const { name, ...others } = { name: "Asha", age: 25, city: "Chennai" };
const [head, ...tail] = [10, 20, 30];

console.log(b);
console.log(older, user);
console.log(sum(1, 2, 3));
console.log(others);
console.log(head, tail);
```

OUTPUT / WHAT HAPPENS

```
9
[ 1, 2, 3, 4 ]
{ name: 'Asha', age: 26 } { name: 'Asha', age: 25 }
6
{ age: 25, city: 'Chennai' }
10 [ 20, 30 ]
```

The original `user` did **not** change. That is the whole point.

COMMON MISTAKES

- **Thinking spread is a deep copy.** It copies only the first level.

```
// ❌ Wrong: inner object is shared
const p = { name: "A", address: { city: "X" } };
const q = { ...p };
q.address.city = "Y";   // p.address.city is also "Y" now!

// ✅ Right: copy the inner object too
const q2 = { ...p, address: { ...p.address, city: "Y" } };
```

- **Changing the original instead of copying.**

```
// ❌ Wrong: mutates the original array
const list = [1, 2];
list.push(3);

// ✅ Right: make a new array
const list2 = [...list, 4];
```

MINI PRACTICE TASK

Start with `const todo = { text: "Study", done: false }`. Create a new object where `done` is `true` without changing `todo`.

KEY TAKEAWAY

> `...` spreads things out (to copy) or gathers leftovers (rest). Copy, don't mutate.

0.6 Array method: `map`

ELI5

A **chocolate factory belt**. Every plain biscuit goes in and comes out wrapped in chocolate. You get the **same number of items**, each one changed.

REAL EXPLANATION

`array.map(fn)` runs `fn` on **every item** and returns a **new array** with the results. The new array always has the **same length**. The original is not changed.

In React, `map` turns a list of data into a list of UI elements. This is the most-used array method in React.

EXAMPLE CODE

```
const prices = [100, 250, 400];

// Make a new array with 10% tax added
const withTax = prices.map((p) => p * 1.1);

// map gives you the item and its index
const labels = prices.map((p, i) => `Item ${i + 1}: ${p}`);

// Turn data into "UI-like" strings (React will return JSX here later)
const names = ["Asha", "Ravi"];
const html = names.map((n) => `<li>${n}</li>`).join("");

console.log(withTax);
console.log(labels);
console.log(html);
console.log(prices); // original is unchanged
```

OUTPUT / WHAT HAPPENS

```
[ 110.00000000000001, 275, 440.00000000000006 ]
[ 'Item 1: 100', 'Item 2: 250', 'Item 3: 400' ]
<li>Asha</li><li>Ravi</li>
[ 100, 250, 400 ]
```

(The long decimals are normal floating-point behaviour. Use `.toFixed(2)` when you show money.)

COMMON MISTAKES

- **Forgetting to return.**

```
// ❌ Wrong: braces but no return -> [undefined, undefined]
const r1 = [1, 2].map((n) => { n * 2 });

// ✅ Right
const r2 = [1, 2].map((n) => n * 2);
```

- **Using `map` just for side effects.** If you don't use the new array, use `forEach`.

```
// ❌ Wrong
[1, 2].map((n) => console.log(n));

// ✅ Right
[1, 2].forEach((n) => console.log(n));
```

MINI PRACTICE TASK

Given `["a", "b", "c"]`, use `map` to return `["A", "B", "C"]` (hint: `.toUpperCase()`).

KEY TAKEAWAY

> `map` changes every item and gives back a new array of the same length.

0.7 Array method: `filter`

ELI5

A **sieve in the kitchen**. You pour in rice and stones. Only the rice passes through. Whatever **passes the test** is kept.

REAL EXPLANATION

`array.filter(fn)` calls `fn` on each item. If `fn` returns something **truthy**, the item is kept. You get a **new array** that can be **shorter** (or even empty). The original is unchanged.

In React, `filter` is how you **delete an item from state** (keep everything except one) and how you build **search boxes**.

EXAMPLE CODE

```
const todos = [
  { id: 1, text: "Study", done: true },
  { id: 2, text: "Cook", done: false },
  { id: 3, text: "Walk", done: false },
];

// Keep only pending todos
const pending = todos.filter((t) => !t.done);

// "Delete" todo with id 2: keep every todo that is NOT id 2
const afterDelete = todos.filter((t) => t.id !== 2);

// Search: items whose text includes "al"
const found = todos.filter((t) => t.text.toLowerCase().includes("al"));

console.log(pending.map((t) => t.text));
console.log(afterDelete.map((t) => t.id));
console.log(found.map((t) => t.text));
```

OUTPUT / WHAT HAPPENS

```
[ 'Cook', 'Walk' ]
[ 1, 3 ]
[ 'Walk' ]
```

COMMON MISTAKES

- **Expecting a single item.** `filter` always gives an **array**.

```
// ❌ Wrong: result is an array, not an object
const todo = todos.filter((t) => t.id === 2);
console.log(todo.text);   // undefined

// ✅ Right: use find for one item
const todo2 = todos.find((t) => t.id === 2);
```

- **Mutating with `splice` in React state.** Use `filter` to get a new array.

```
// ❌ Wrong
todos.splice(1, 1);

// ✅ Right
const next = todos.filter((t) => t.id !== 2);
```

MINI PRACTICE TASK

From `[5, 12, 8, 20]`, keep only numbers greater than 10.

KEY TAKEAWAY

> `filter` keeps the items that pass your test and returns a new array.

0.8 Array method: `reduce`

ELI5

You have a **piggy bank** and a pile of coins. You pick up one coin at a time and drop it in. At the end, the whole pile has become **one number**: your total. `reduce` squashes many things into one.

REAL EXPLANATION

`array.reduce(fn, start)` walks through the array. `fn(accumulator, item)` gets the running result (the **accumulator**) and the current item. Whatever `fn` returns becomes the accumulator for the next round. The final accumulator is the answer.

The answer can be a number, an object, an array, anything. Always give a **start value** (second argument).

```
 start = 0
   [10, 20, 30]
    |
    +-- acc=0  + 10  -> 10
    +-- acc=10 + 20  -> 30
    +-- acc=30 + 30  -> 60   <-- final answer
```

In React, `reduce` is used for totals (cart price), counting, grouping, and inside `useReducer` (Part 3).

EXAMPLE CODE

```
const cart = [
  { name: "Pen", price: 10, qty: 3 },
  { name: "Book", price: 120, qty: 1 },
  { name: "Bag", price: 500, qty: 2 },
];

// 1. Total price: many items -> one number
const total = cart.reduce((sum, item) => sum + item.price * item.qty, 0);

// 2. Count how many times each word appears: many items -> one object
const words = ["a", "b", "a", "c", "a"];
const counts = words.reduce((acc, w) => {
  acc[w] = (acc[w] || 0) + 1;
  return acc; // always return the accumulator
}, {});

console.log(total);
console.log(counts);
```

OUTPUT / WHAT HAPPENS

```
1150
{ a: 3, b: 1, c: 1 }
```

COMMON MISTAKES

- **No start value.** With an empty array, `reduce` crashes.

```
// ❌ Wrong: TypeError on an empty array
[].reduce((a, b) => a + b);

// ✅ Right
[].reduce((a, b) => a + b, 0);   // 0
```

- **Forgetting to return the accumulator.**

```
// ❌ Wrong: acc becomes undefined on the next round
words.reduce((acc, w) => { acc[w] = 1; }, {});

// ✅ Right
words.reduce((acc, w) => { acc[w] = 1; return acc; }, {});
```

MINI PRACTICE TASK

Use `reduce` to find the **largest** number in `[3, 9, 4]`. (Hint: start with `-Infinity`, or with the first item.)

KEY TAKEAWAY

> `reduce` folds a whole array into one value. Always give a start value and always return the accumulator.

0.9 Array method: `find` (and friends)

ELI5

You are looking for **your red socks** in a drawer. You stop the moment you see them. You don't check the rest. `find` stops at the **first match**.

REAL EXPLANATION

Method	Returns	Stops early?
`find(fn)`	the **first item** that passes, or `undefined`	Yes
`findIndex(fn)`	the **position** of the first match, or `-1`	Yes
`some(fn)`	`true` if **at least one** passes	Yes
`every(fn)`	`true` if **all** pass	Yes
`includes(x)`	`true` if the value is in the array	Yes

In React you use `find` to pick one item by id (for a detail page) and `some`/`every` for checks like "is any field empty?".

EXAMPLE CODE

```
const users = [
  { id: 1, name: "Asha", admin: false },
  { id: 2, name: "Ravi", admin: true },
  { id: 3, name: "Meena", admin: false },
];

const ravi = users.find((u) => u.id === 2);          // the object
const missing = users.find((u) => u.id === 99);      // undefined
const index = users.findIndex((u) => u.name === "Meena");
const hasAdmin = users.some((u) => u.admin);         // true
const allAdmin = users.every((u) => u.admin);        // false
const hasTwo = [1, 2, 3].includes(2);                // true

console.log(ravi);
console.log(missing);
console.log(index, hasAdmin, allAdmin, hasTwo);
```

OUTPUT / WHAT HAPPENS

```
{ id: 2, name: 'Ravi', admin: true }
undefined
2 true false true
```

COMMON MISTAKES

- **Using the result without checking for `undefined`.**

```
// ❌ Wrong: crashes if user 99 doesn't exist
console.log(users.find((u) => u.id === 99).name);

// ✅ Right
const u = users.find((x) => x.id === 99);
console.log(u ? u.name : "Not found");
// or: users.find((x) => x.id === 99)?.name   (see 0.13)
```

- **Using `filter(...)[0]` when you want one item.** `find` is simpler and faster.

MINI PRACTICE TASK

From `["red", "green", "blue"]`, check if it includes `"green"`, and find the first color whose length is greater than 3.

KEY TAKEAWAY

> `find` gets the first match, `some`/`every` answer yes/no questions. Always handle “not found”.

0.10 Modules: `import` and `export`

ELI5

A big school has **many classrooms**. Each classroom (file) keeps its own stuff. If one class wants to use another class's globe, the owner must **offer it** (`export`) and the other class must **ask for it** (`import`).

REAL EXPLANATION

A **module** is just a file. Variables inside are private unless you `export` them. Other files `import` what they need.

Type	Export	Import	How many per file
Named	`export const add = ...`	`import { add } from "./math.js"`	many
Default	`export default Button`	`import Button from "./Button.js"`	one

- Named imports must use the **same name** (you may rename with `as`).
- Default imports can use **any name**.
- A path starting with `./` means "this folder". A bare name like `"react"` means "an installed package".
- **React: one component per file, usually a default export.** Helpers and hooks use named exports.

```
math.mjs  ---- export add, PI ---->  main.mjs
                                       import { add, PI } from "./math.mjs"
```

EXAMPLE CODE

Save these two files in the same folder. Use the `.mjs` extension so Node treats them as modules.

```js
// ===== File 1: math.mjs =====
export const PI = 3.14;                    // named export
export const add = (a, b) => a + b;        // named export
export default function greet(name) {      // default export (only one)
  return `Hello, ${name}!`;
}
```

```js
// ===== File 2: main.mjs =====
// Run with: node main.mjs
import greet, { add, PI as pie } from "./math.mjs";
// "greet" is the default import (any name works)
// { add, PI as pie } are named imports (PI renamed to pie)

console.log(greet("Asha"));
console.log(add(2, 3));
console.log(pie);
```

OUTPUT / WHAT HAPPENS

```
Hello, Asha!
5
3.14
```

COMMON MISTAKES

- **Curly braces on a default import (and none on a named one).**

```
// ❌ Wrong
import { greet } from "./math.mjs";   // greet is a default export
import add from "./math.mjs";         // add is a named export

// ✅ Right
import greet, { add } from "./math.mjs";
```

- **Forgetting `./` for your own files.**

```
// ❌ Wrong: looks for an npm package called "math.mjs"
import { add } from "math.mjs";

// ✅ Right
import { add } from "./math.mjs";
```

MINI PRACTICE TASK

Create `shape.mjs` that exports a function `area(w, h)`. Import and use it in `main.mjs`.

KEY TAKEAWAY

> `export` offers, `import` asks. Named = with `{ }`, default = without.

0.11 Promises

ELI5

You order a **pizza** and get a **buzzer**. The buzzer is a promise: “pizza is coming”. While you wait you can chat. Soon it either **rings** (pizza ready = success) or **blinks red** (oven broke = failure).

REAL EXPLANATION

Some work takes time (loading data from a server, reading a file). JavaScript does not freeze while waiting. It gives you a **Promise**: an object that will have a result **later**.

A promise is always in one of 3 states:

```
          +--> fulfilled (success, has a value) --> .then(value => ...)
pending --+
          +--> rejected  (failure, has an error) --> .catch(error => ...)
```

- `.then(fn)` runs when it succeeds.
- `.catch(fn)` runs when it fails.
- `.finally(fn)` runs either way.
- You can chain `.then` calls. Each one gets the previous one's return value.
- `fetch()` in React returns a promise. So does almost every API call.

EXAMPLE CODE

```
// A fake server call: waits 500 ms, then succeeds or fails
function fakeFetchUser(id) {
  return new Promise((resolve, reject) => {
    setTimeout(() => {
      if (id > 0) resolve({ id, name: "Asha" }); // success
      else reject(new Error("Invalid id"));      // failure
    }, 500);
  });
}

console.log("1. Asking for user...");

fakeFetchUser(1)
  .then((user) => {
    console.log("3. Got user:", user.name);
    return user.id * 100;          // passed to the next .then
  })
  .then((n) => console.log("4. Next step got:", n))
  .catch((err) => console.log("Error:", err.message))
  .finally(() => console.log("5. Done (success or fail)"));

console.log("2. This line runs while we wait!");
```

OUTPUT / WHAT HAPPENS

```
1. Asking for user...
2. This line runs while we wait!
3. Got user: Asha
4. Next step got: 100
5. Done (success or fail)
```

Line 2 prints **before** line 3. JavaScript did not wait. After about half a second the promise finished and the `.then` code ran. Change `fakeFetchUser(1)` to `fakeFetchUser(-1)` to see the `.catch` path.

COMMON MISTAKES

- **Treating a promise like the final value.**

```
// ❌ Wrong: user is a Promise, not an object
const user = fakeFetchUser(1);
console.log(user.name);   // undefined

// ✅ Right
fakeFetchUser(1).then((u) => console.log(u.name));
```

- **Forgetting `.catch`.** An unhandled rejection can crash Node or fail silently in the browser.

```
// ❌ Wrong
fakeFetchUser(-1).then((u) => console.log(u));

// ✅ Right
fakeFetchUser(-1).then((u) => console.log(u)).catch((e) => console.log(e.message));
```

- **Not returning inside `.then`.** The next `.then` gets `undefined`.

MINI PRACTICE TASK

Write `wait(ms)` that returns a promise resolving after `ms` milliseconds. Use `.then` to print `"done"` after 1 second.

KEY TAKEAWAY

> A promise is a “result coming later”. Use `.then` for success and `.catch` for failure.

0.12 async / await

ELI5

Same pizza buzzer, but now you write the story **like a normal list**: “order pizza, wait for the buzzer, eat pizza”. The word `await` means “pause *me* here until the buzzer rings”. Other people in the shop keep working.

REAL EXPLANATION

`async`/`await` is a cleaner way to write promise code. It reads top-to-bottom like normal code.

- Put `async` before a function. That function now **always returns a promise**.
- Inside it, `await promise` pauses **that function** until the promise settles, then gives you the value.
- Use `try / catch / finally` for errors instead of `.catch`.
- `await` only works inside an `async` function (or at the top level of a module).
- To run things **at the same time**, use `Promise.all([...])`.

In React you write `async` functions for every API call (but **not** as the component function itself, which you will see in Part 3).

EXAMPLE CODE

```
const wait = (ms) => new Promise((resolve) => setTimeout(resolve, ms));

async function getUser(id) {
  await wait(300);
  if (id <= 0) throw new Error("Invalid id");
  return { id, name: "User" + id };
}

async function main() {
  try {
    // Sequential: one after another (about 600 ms)
    const a = await getUser(1);
    const b = await getUser(2);
    console.log("Sequential:", a.name, b.name);

    // Parallel: both at the same time (about 300 ms)
    const [c, d] = await Promise.all([getUser(3), getUser(4)]);
    console.log("Parallel:", c.name, d.name);

    await getUser(-1); // this one fails
    console.log("never printed");
  } catch (err) {
    console.log("Caught:", err.message);
  } finally {
    console.log("Finished");
  }
}

main();
```

OUTPUT / WHAT HAPPENS

```
Sequential: User1 User2
Parallel: User3 User4
Caught: Invalid id
Finished
```

The sequential lines take about 0.6 seconds. The parallel pair takes about 0.3 seconds, because both waited together.

COMMON MISTAKES

- **Forgetting `await`.**

```
// ❌ Wrong: user is a Promise
const user = getUser(1);
console.log(user.name);   // undefined

// ✅ Right
const user2 = await getUser(1);
```

- **Using `await` in a normal function.**

```
// ❌ Wrong: SyntaxError
function load() { const u = await getUser(1); }

// ✅ Right
async function load2() { const u = await getUser(1); }
```

- **Awaiting one by one when they don't depend on each other.** Use `Promise.all`.

MINI PRACTICE TASK

Write an `async` function that waits 1 second, then returns `"Hello"`. Call it and print the result.

KEY TAKEAWAY

> `await` pauses an `async` function until a promise is done. Use `try/catch` for errors and `Promise.all` for parallel work.

0.13 Optional chaining `?.`

ELI5

You want to open **a door, then a cupboard behind it, then take the cookie**. If the door isn't there, you don't want to crash into a wall. `?.` says: **"if this thing exists, keep going; if not, stop and say 'nothing'."**

REAL EXPLANATION

Reading a property of `undefined` or `null` throws an error: `Cannot read properties of undefined`. This is the **most common error in React apps**, because data often arrives late (after an API call).

`?.` stops safely and returns `undefined` instead of crashing.

Syntax	Meaning
`obj?.prop`	read `prop` only if `obj` is not null/undefined
`obj?.[key]`	same, with brackets
`fn?.()`	call `fn` only if it exists
`arr?.[0]`	read first item only if `arr` exists

EXAMPLE CODE

```
const user = {
  name: "Asha",
  address: { city: "Chennai" },
  greet() { return "Hi!"; },
};
const empty = null;

console.log(user.address?.city);       // "Chennai"
console.log(user.job?.title);          // undefined (no job, no crash)
console.log(empty?.name);              // undefined (no crash)
console.log(user.greet?.());           // "Hi!"
console.log(user.dance?.());           // undefined (no function, no crash)

const list = undefined;
console.log(list?.[0]);                // undefined

// Without ?. this line would crash:
// console.log(user.job.title);
```

OUTPUT / WHAT HAPPENS

```
Chennai
undefined
undefined
Hi!
undefined
undefined
```

COMMON MISTAKES

- **Using `?.` everywhere "just in case".** It can hide real bugs. Use it only where data may truly be missing.

```
// ❌ Wrong: user always exists here, so ?. hides mistakes
const n = user?.name;

// ✅ Right: use it for data that may be missing
const t = user.job?.title;
```

- **Using `?.` on the left side of an assignment.**

```
// ❌ Wrong: SyntaxError
user?.name = "Ravi";

// ✅ Right
if (user) user.name = "Ravi";
```

MINI PRACTICE TASK

Make `const data = { profile: null }`. Safely print `data.profile?.email`.

KEY TAKEAWAY

> `?.` means “only continue if it exists”. It prevents “Cannot read properties of undefined”.

0.14 Nullish coalescing `??`

ELI5

“If you don’t have a pencil, **use this spare one**.” But the important part: if you have a pencil that is **0 cm long**, it is still *your pencil*. It is only “missing” when you truly have nothing (`null` or `undefined`).

REAL EXPLANATION

`a ?? b` returns `b` **only if** `a` **is** `null` **or** `undefined`. Otherwise it returns `a`.

It differs from `||`, which replaces **every falsy value**: `0`, `""`, `false`, `NaN`. That is often a bug.

`a` is…	**`a || "x"`**	**`a ?? "x"`**
`undefined`	`"x"`	`"x"`
`null`	`"x"`	`"x"`
`0`	`"x"` (!)	`0`
`""`	`"x"` (!)	`""`
`false`	`"x"` (!)	`false`
`"hi"`	`"hi"`	`"hi"`

It pairs well with `?.`: `user.nickname ?? "Guest"`. There is also `??=` which assigns only when the value is missing.

EXAMPLE CODE

```
const settings = { volume: 0, nickname: "", theme: undefined };

// || treats 0 and "" as missing (usually wrong)
console.log(settings.volume || 50);       // 50  (bug: user chose 0!)
console.log(settings.nickname || "Guest");// "Guest"

// ?? only replaces null / undefined (correct)
console.log(settings.volume ?? 50);       // 0
console.log(settings.nickname ?? "Guest");// ""
console.log(settings.theme ?? "light");   // "light"

// Combine with ?.
const user = { profile: null };
console.log(user.profile?.city ?? "Unknown city");

// ??= assigns only if missing
let color;
color ??= "blue";
console.log(color);
```

OUTPUT / WHAT HAPPENS

```
50
Guest
0
(empty line)
light
Unknown city
blue
```

The 4th line prints an empty string, so you see a blank line.

COMMON MISTAKES

- **Using `||` for defaults when `0` or `""` are valid.**

```
// ❌ Wrong: a score of 0 becomes 100
const score = 0;
const shown = score || 100;

// ✅ Right
const shown2 = score ?? 100;
```

- **Mixing `??` with `||` or `&&` without parentheses.** It is a SyntaxError.

```
// ❌ Wrong
const v = a || b ?? c;

// ✅ Right
const v2 = (a || b) ?? c;
```

MINI PRACTICE TASK

Write `const age = 0`. Print the age using `??` so that `0` stays `0`, and print `"unknown"` for `undefined`.

KEY TAKEAWAY

> `??` gives a default only for `null`/`undefined`. Use it instead of `||` when `0`, `""` or `false` are real values.

Part 0 Quiz

Try all 5 before reading the answers.

1. Which line throws an error? `const a = [1]; a.push(2);` or `const b = 1; b = 2;`
2. What does `[1, 2, 3].map((n) => { n * 2 })` return?
3. What is the result of `0 ?? 10` and of `0 || 10`?
4. How do you wait for two independent API calls **at the same time**?
5. A file has `export default function Card() {}`. Write the correct import line.

Quiz answers

1. `b = 2` throws, because `b` is a `const`. `a.push(2)` is fine: `const` protects the label, not the contents.
2. `[undefined, undefined, undefined]`. The braces need a `return`.
3. `0 ?? 10` is `0`. `0 || 10` is `10`.
4. `const [x, y] = await Promise.all([callA(), callB()]);`
5. `import Card from "./Card.js";` (no curly braces for a default export).

Part 0 Mini Project: Shopping Cart Report

Goal: One small program that uses almost every topic in this Part.

You have a product list from a “server”. Build a report: load data (promise + `async/await`), filter in-stock items, add tax (`map`), compute totals (`reduce`), find the cheapest (`find`/`reduce`), and print safe text (`?.`, `??`, template literals, destructuring, spread).

Save as `cart.mjs` and run `node cart.mjs`.

```
// cart.mjs  —  Part 0 mini project
// ---------- 1. Fake server (promises, async/await) ----------
const wait = (ms) => new Promise((resolve) => setTimeout(resolve, ms));

async function fetchProducts() {
  await wait(300); // pretend we call an API
  return [
    { id: 1, name: "Pen", price: 10, qty: 10, inStock: true, discount: 0 },
    { id: 2, name: "Notebook", price: 60, qty: 2, inStock: true },
    { id: 3, name: "Bag", price: 500, qty: 1, inStock: false },
    { id: 4, name: "Bottle", price: 150, qty: 3, inStock: true, discount: 10 },
  ];
}

// ---------- 2. Helpers (arrow functions) ----------
const TAX_RATE = 0.18; // const: never changes
const money = (n) => `₹${n.toFixed(2)}`; // template literal

// Price of one line after discount (?? gives 0 when discount is missing)
const lineTotal = ({ price, qty, discount }) =>
  price * qty * (1 - (discount ?? 0) / 100);

// ---------- 3. Main program ----------
async function main() {
  const products = await fetchProducts();

  // filter: only items in stock
  const available = products.filter((p) => p.inStock);

  // map: add a "total" to each item (spread copies the object)
  const lines = available.map((p) => ({ ...p, total: lineTotal(p) }));

  // reduce: add all totals
  const subtotal = lines.reduce((sum, l) => sum + l.total, 0);
  const tax = subtotal * TAX_RATE;
  const grandTotal = subtotal + tax;

  // reduce: find the cheapest line
  const cheapest = lines.reduce((min, l) => (l.total < min.total ? l : min));

  // find + ?. : look up an id that may not exist
  const item9 = products.find((p) => p.id === 9);

  // destructuring in the loop
  console.log("=== Cart Report ===");
  for (const { name, qty, total } of lines) {
    console.log(`${name} x${qty} = ${money(total)}`);
  }
  console.log("--------------------");
  console.log(`Subtotal: ${money(subtotal)}`);
  console.log(`Tax (18%): ${money(tax)}`);
  console.log(`Grand total: ${money(grandTotal)}`);
  console.log(`Cheapest line: ${cheapest.name}`);
  console.log(`Item 9: ${item9?.name ?? "not found"}`);
}

main().catch((err) => console.log("Failed:", err.message));
```

Expected output

```
=== Cart Report ===
Pen x10 = ₹100.00
Notebook x2 = ₹120.00
Bottle x3 = ₹405.00
-------------------
Subtotal: ₹625.00
Tax (18%): ₹112.50
Grand total: ₹737.50
Cheapest line: Pen
Item 9: not found
```

Stretch goals

1. Move the helpers into `helpers.mjs` and `import` them.
2. Add a `searchTerm` and filter lines by name.
3. Use `Promise.all` to fetch products and a “coupons” list together.

Topics used: 0.1 `const`/`let` • 0.2 arrows • 0.3 templates • 0.4 destructuring • 0.5 spread • 0.6 `map` • 0.7 `filter` • 0.8 `reduce` • 0.9 `find` • 0.11–0.12 promises and `async/await` • 0.13 `?.` • 0.14 `??`

---

*End of Part 0. Say **“next”** to continue to Part 1 – React Basics.*